Container Escape

A Comprehensive Security Analysis of Kubernetes Cluster Compromise Using Attack Tree Methodology

Malcolm Odubote, Ridwan Abdulwaheed, and Honour Sigbeku

Memorial University of Newfoundland

Security and Privacy in Computer Systems

Instructor: Dr. Yashar Tavakoli

Abstract

Kubernetes has become the dominant container orchestration platform, with 96% of organizations either using or evaluating it for critical infrastructure. However, its default configuration lacks essential security hardening, making misconfiguration the leading cause of container-related breaches. This report investigates Kubernetes security through the perspective of traditional security frameworks, that is, the CIA triad, threat taxonomy, attack surface analysis, fundamental security design principles, and FIPS 200 security requirements, which demonstrate that these foundational concepts map directly to modern container orchestration challenges.

Using attack tree methodology, the Microsoft Threat Matrix for Kubernetes, and the MITRE ATT&CK for Containers framework, we construct a comprehensive threat model and identify a specific attack path: RBAC misconfiguration enabling credential theft and full cluster takeover. Through a practical demonstration, we execute this attack against a Kubernetes cluster, achieving complete secret exfiltration through an overly permissive ServiceAccount. We then apply a layered defence strategy guided by CIS Kubernetes Benchmarks, showing that enforcing a single principle, least privilege, blocks the entire attack chain. The results demonstrate that Kubernetes provides well-defined security mechanisms, but the gap between secure by design and secure by default remains a challenge requiring intentional configuration and continuous assessment.

Keywords: *Kubernetes, container escape, RBAC, attack tree, least privilege, CIS benchmark, MITRE ATT&CK*

1. Introduction

Container technology has fundamentally changed how organizations deploy and manage applications. Rather than running software directly on servers, modern applications are packaged into containers, which are isolated, lightweight units that bundle an application with everything it needs to run. When an organization operates hundreds or thousands of these containers, managing them manually becomes impossible. Kubernetes emerged as the solution: an open-source container orchestrator that automates deployment, scaling, restart, and secret management across clusters of machines.

The adoption has been remarkable. Industry surveys report that 96% of organizations are either using or evaluating Kubernetes, and it now underpins critical infrastructure across banking, healthcare, and government sectors (Matt Li, 2025). However, Kubernetes is not secure by default. It requires intentional hardening before production use (Reza Ramezanpour, 2025), and misconfigurations remain the leading cause of container-related breaches (Sarika Sharma, 2025).

The consequences of these misconfigurations are not theoretical. In 2018, Tesla's Kubernetes dashboard was left publicly accessible, allowing attackers to hijack cluster resources for cryptocurrency mining. In 2019, a misconfigured access control policy at Capital One led to the exposure of over 100 million customer records (ShiftLeft Blog, 2025). As recently as 2025, an ingress misconfiguration in the HAProxy Kubernetes Ingress Controller enabled secret leakage and privilege escalation (HAProxy Technologies, 2025). Each of these incidents traces back to the same root cause: security mechanisms existed but were not properly configured.

This report addresses three research questions. First, how do fundamental security concepts apply to Kubernetes? Second, what attack paths lead from an initial foothold inside a container to full cluster compromise? Third, how effectively can layered security controls defend against these attacks? To answer these, attack tree methodology is applied using the Microsoft Threat Matrix for Kubernetes and the MITRE ATT&CK for Containers framework. A specific attack is then demonstrated, exploiting RBAC misconfiguration to steal credentials and exfiltrate cluster secrets, before showing how CIS Kubernetes Benchmarks and the least privilege principle transform the outcome from full compromise to access denied.

The report is structured as follows. Section 2 provides background on Kubernetes architecture. Section 3 maps traditional security frameworks to Kubernetes. Section 4 presents the attack methodology and threat modelling. Section 5 details the practical demonstration. Section 6 discusses the security strategy. Section 7 presents conclusions.

2. Background: Kubernetes Architecture

To understand how Kubernetes can be compromised, it is first necessary to understand how it is structured. This section introduces the core components relevant to the security analysis that follows.

2.1 Containers and Orchestration

A container is an isolated runtime environment that packages an application together with its dependencies, libraries, and configuration files. Containers share the host operating system's kernel, which makes them lightweight and fast to start. A typical production environment may run hundreds or thousands of containers simultaneously, and managing them at that scale introduces a coordination problem: containers need to be started, monitored, restarted when they fail, scaled up under heavy load, and scaled down when demand drops. Kubernetes solves this as an automated system that manages the entire lifecycle of containerized applications across a cluster of machines.

2.2 Cluster Architecture

A Kubernetes cluster consists of two logical layers: the control plane, which makes decisions about the cluster, and the worker nodes, which run the application containers. Figure 1 illustrates the key components and how they relate.

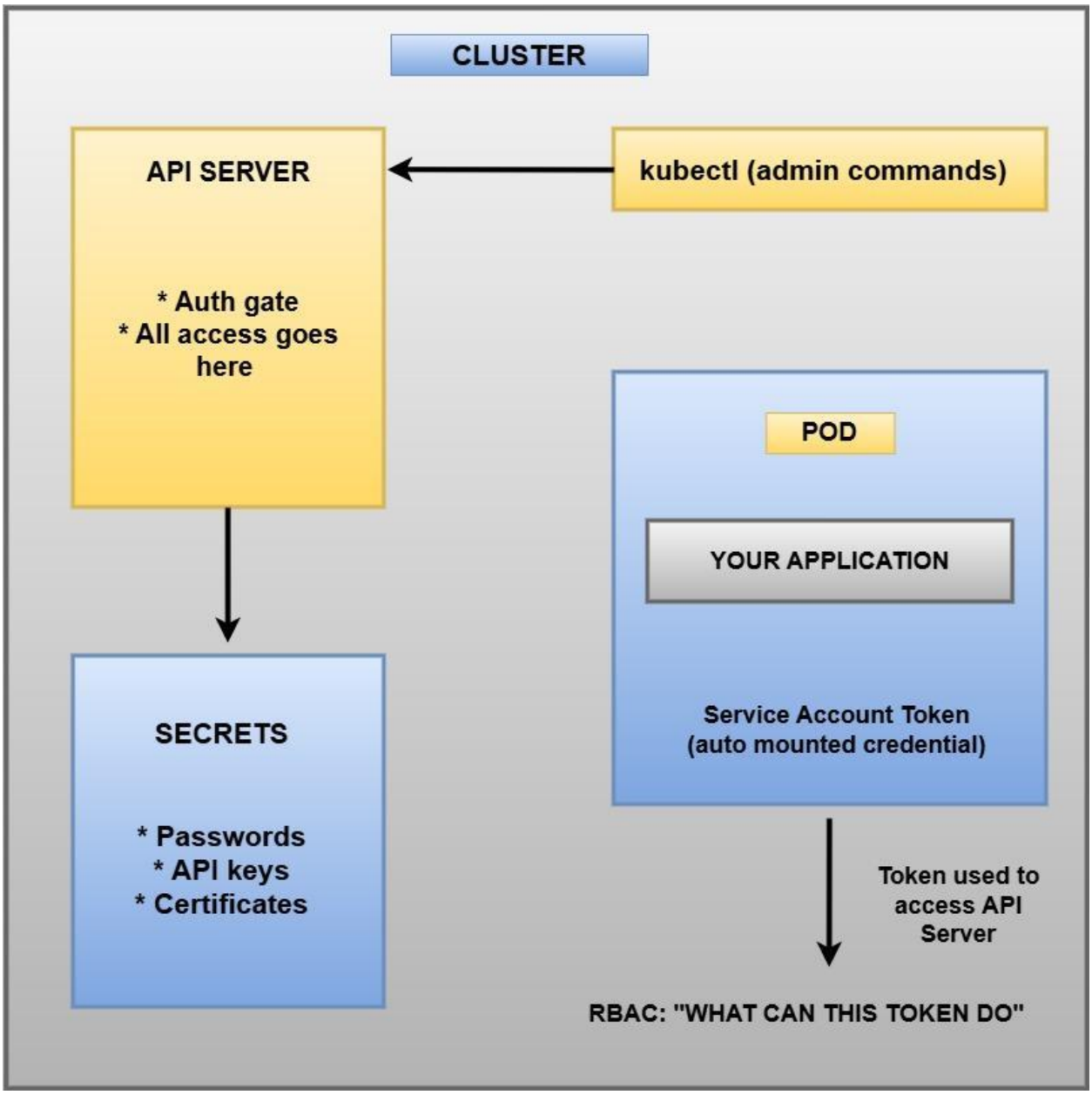

Figure 1. Kubernetes cluster architecture: API Server, Secrets, Pod, Service Account Token, and RBAC.

The central component is the API Server. Every operation in the cluster must pass through it and be authenticated. Behind it is etcd, a distributed key-value store holding the entire cluster state including every secret and configuration. Compromising etcd is equivalent to compromising the entire cluster, as it is "the authoritative record" of what should be running (Guarino, 2026). On the worker nodes, the Kubelet agent ensures the correct containers are running inside Pods, which are the smallest deployable unit in Kubernetes.

2.3 Authentication and Access Control

Kubernetes uses Service Accounts to provide identities for processes running inside Pods. By default, every Pod is automatically mounted with a Service Account token for authenticating against the API Server. What the token is permitted to do is determined by Role-Based Access Control (RBAC), which operates on a default-deny model: a Service Account has no permissions unless explicitly granted. Permissions are defined in Roles or ClusterRoles and attached through RoleBindings or ClusterRoleBindings. The security problem arises when administrators grant excessive privileges.

2.4 Why This Is Relevant

This architecture reveals why Kubernetes security is fundamentally an access control problem. The API Server controls all access, tokens are automatically provided to every Pod, and RBAC determines what those tokens can do. If RBAC is misconfigured, the entire trust model breaks down.

3. Theory: Foundational Security Concepts Applied to Kubernetes

A central argument of this report is that foundational security concepts, though developed before container orchestration existed, remain directly applicable to current-generation infrastructure.

This section maps each concept to Kubernetes.

3.1 Asset Classification

"The assets of a computer system can be categorized as: Hardware, Software, Data, and Communication lines and networks" (Tavakoli, 2026). Table 1 maps these categories onto Kubernetes.

ASSET TYPE	KUBERNETES COMPONENTS
HARDWARE	Nodes (physical/virtual machines) Storage hardware
SOFTWARE	API Server, Kubelet, Kube-scheduler, container runtime
DATA	etcd database, Secrets, ConfigMaps, Container images
COMMUNICATION	API server endpoints, Pod-to-pod network, Ingress/Egress traffic

Table 1. Kubernetes asset classification: hardware (worker/master nodes), software (API Server, Kubelet, etcd), data (secrets, ConfigMaps), and communication (API endpoints, pod network).

Identifying these assets determines what needs protection. So, If an organization fails to recognize etcd as a data asset, it may neglect encrypting it or restricting access to it, which could have severe consequences.

3.2 CIA Triad Analysis

With assets identified, Table 2 assesses the impact of compromise across the CIA triad for each key component.

Component	Confidentiality	Integrity	Availability
API Server	**High**	**High**	**High**
etcd Database	**High**	**High**	**High**
Worker Nodes	**Medium**	**High**	**High**
Service Networking	**Medium**	**High**	**High**

Table 2. CIA triad impact analysis for Kubernetes components.

The API Server and etcd both carry high impact across all three dimensions. Worker nodes have moderate confidentiality impact since a compromised node exposes only its own containers. Network communication carries moderate confidentiality risk but high integrity and availability impact because modified API requests can change cluster behaviour entirely.

3.3 Threat Classification

"There are four kinds of threats: Unauthorized disclosure, Deception, Disruption, Usurpation" (Tavakoli, 2026). Each appears distinctly in Kubernetes.

Unauthorized disclosure manifests as secrets exposure, service account token theft, or direct etcd access. Deception appears as malicious pod impersonation, injection of falsified credentials, or DNS spoofing. Disruption takes the form of pod deletion, resource exhaustion through cryptocurrency mining as in the Tesla breach, or network flooding. Usurpation is the most severe, encompassing privilege escalation, cluster admin takeover, and container escape. These categories can overlap, as the demonstration in Section 5 shows: an unauthorized disclosure attack enabled by usurpation through a misconfigured Service Account.

3.4 Attack Surface Analysis

"Attack surfaces can be categorized as: Network, Software, and Human" (Tavakoli, 2026). Kubernetes exhibits vulnerabilities across all three. See figure 2 below.

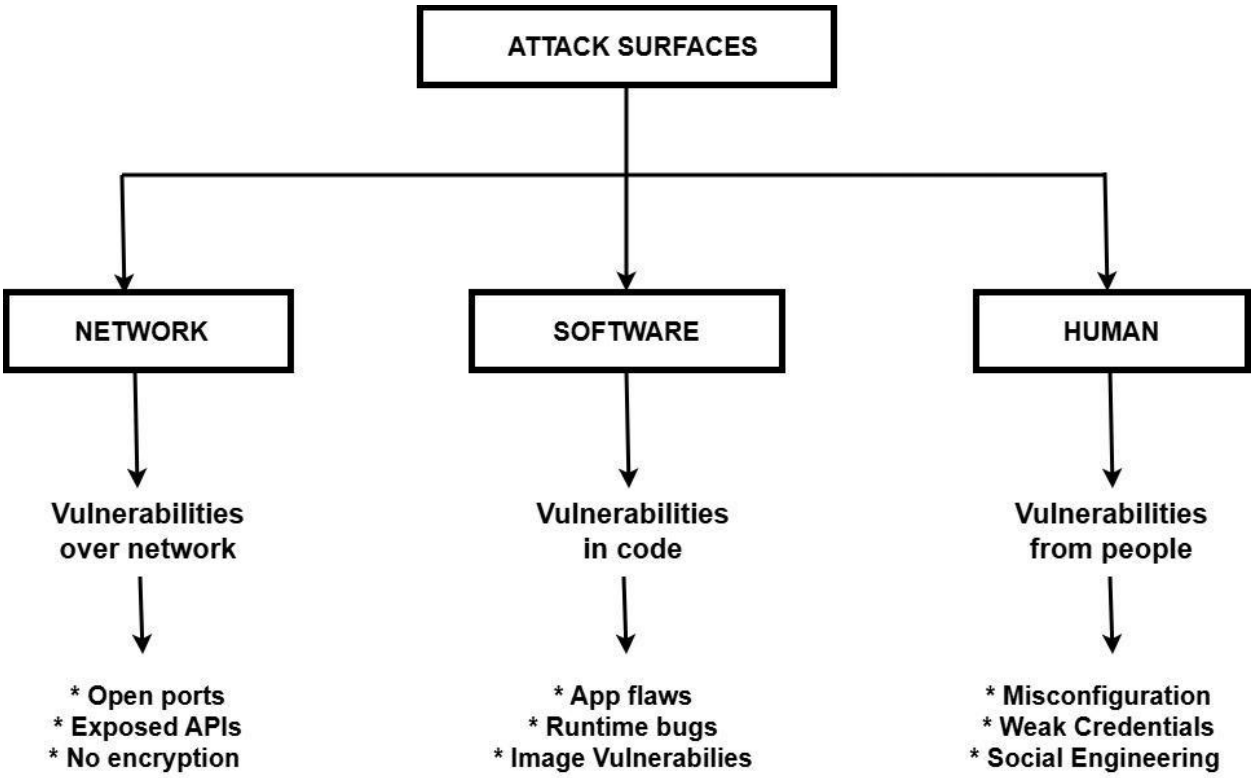

Figure 2: Attack Surfaces mapped to Kubernetes

The network attack surface includes the API Server exposed without authentication, etcd ports 2379 and 2380, absent network policies allowing all pod-to-pod traffic by default, and NodePort services exposing internal services externally. The software attack surface includes container runtime CVEs, Kubelet vulnerabilities, vulnerable base images, and application bugs. The human attack surface, which is where the demonstration focuses, includes RBAC misconfiguration, auto-mounted tokens, secrets hardcoded in application code, and overly permissive defaults left unchanged during deployment.

3.5 Fundamental Security Design Principles

There are thirteen fundamental security design principles for building secure systems. Table 3 evaluates Kubernetes against each one.

Principle	K8s Assessment	Rating
Economy of Mechanism	Complex system; numerous components increase attack surface.	POOR
Fail-Safe Defaults	RBAC denies by default (good), but tokens are auto-mounted (bad).	PARTIAL
Complete Mediation	All requests pass through the API Server.	GOOD
Open Design	Fully open source, auditable, no security through obscurity.	EXCELLENT
Separation of Privilege	RBAC supports multiple roles; admission controllers add layers.	GOOD
Least Privilege	Fine-grained RBAC exists but requires deliberate configuration.	PARTIAL
Least Common Mechanism	Namespaces, network policies, and containers enable isolation.	GOOD
Psych. Acceptability	Some tools are intuitive; security config is still complex.	PARTIAL
Isolation	Multiple isolation layers: containers, namespaces, network policies.	GOOD
Encapsulation	Pods encapsulate containers; services abstract pods.	GOOD
Modularity	Strongly supports microservice and pluggable architecture.	EXCELLENT
Layering	Security implementable at container, pod, network, and cluster levels.	GOOD
Least Astonishment	Well-documented API with consistent behaviour.	GOOD

Table 3. Evaluation of Kubernetes against thirteen fundamental security design principles.

The principle most relevant to the demonstration is least privilege. The attack succeeds entirely because a Service Account was granted cluster-admin permissions when minimal access sufficed. Enforcing this single principle blocks the entire attack chain.

3.6 FIPS 200 Security Requirements

FIPS 200 specifies seventeen security-related areas that federal information systems must address. Table 4 shows how Kubernetes satisfies the key requirements.

FIPS 200 Requirement	Kubernetes Implementation
Access Control	RBAC / ABAC / Service Accounts
Audit and Accountability	API Server audit logs
Security Assessment	kube-bench / CIS Benchmarks
Configuration Management	Pod Security Standards / Admission Controllers
Identification & Authentication	Service Account Tokens, TLS Certificates
System and Information Integrity	Runtime monitoring through Falco
Others (13 requirements)	Depend on the organization

Table 4. FIPS 200 requirements and corresponding Kubernetes implementations.

4. Attack Methodology: Threat Modelling and Attack Tree Construction

This section turns to the practical question: how does an attacker move from an initial foothold inside a container to full cluster compromise? Attack tree methodology is applied using the MITRE ATT&CK for Containers framework.

"An attack tree is a branching, hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities" (Tavakoli, 2026).

4.1 MITRE ATT&CK for Containers

Security teams draw on MITRE ATT&CK, a globally maintained knowledge base cataloguing real-world attack techniques. MITRE ATT&CK for Containers organizes these into tactical phases: Initial Access, Execution, Persistence, Privilege Escalation, Defence Evasion, Credential Access, Discovery, Lateral Movement, and Impact. Figure 3 shows the full matrix (MITRE Corporation, n.d.).

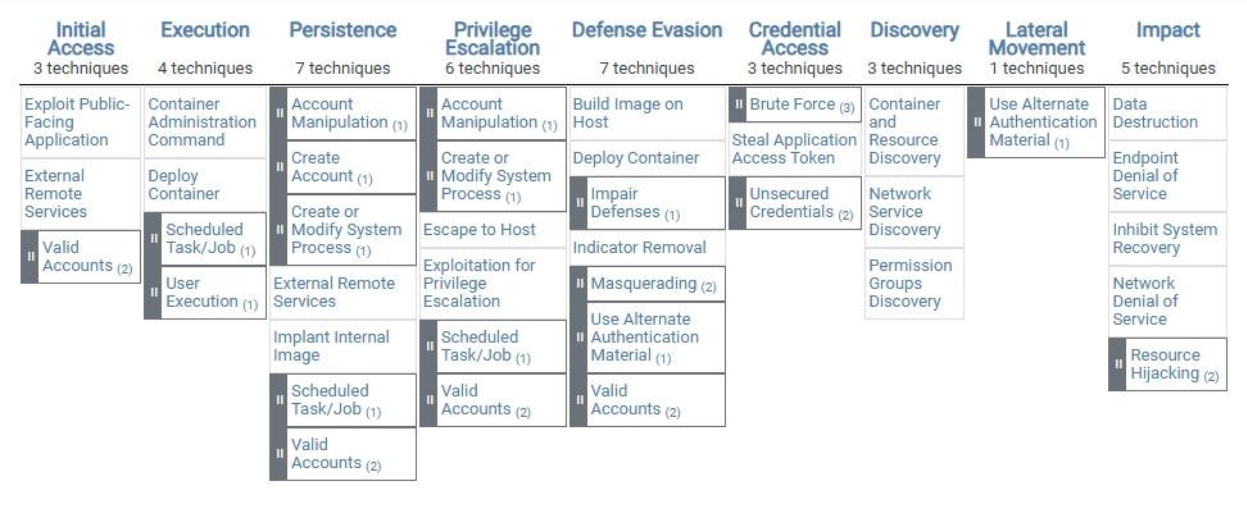

Figure 3. MITRE ATT&CK Matrix for Containers showing all tactical phases and techniques.

4.2 Comprehensive Attack Tree: Kubernetes Cluster Compromise

Using MITRE ATT&CK for Containers and Microsoft's Threat Matrix for Kubernetes (Microsoft, n.d.), a comprehensive attack tree is constructed with the root goal of compromising a Kubernetes cluster. Figure 4 illustrates the structure.

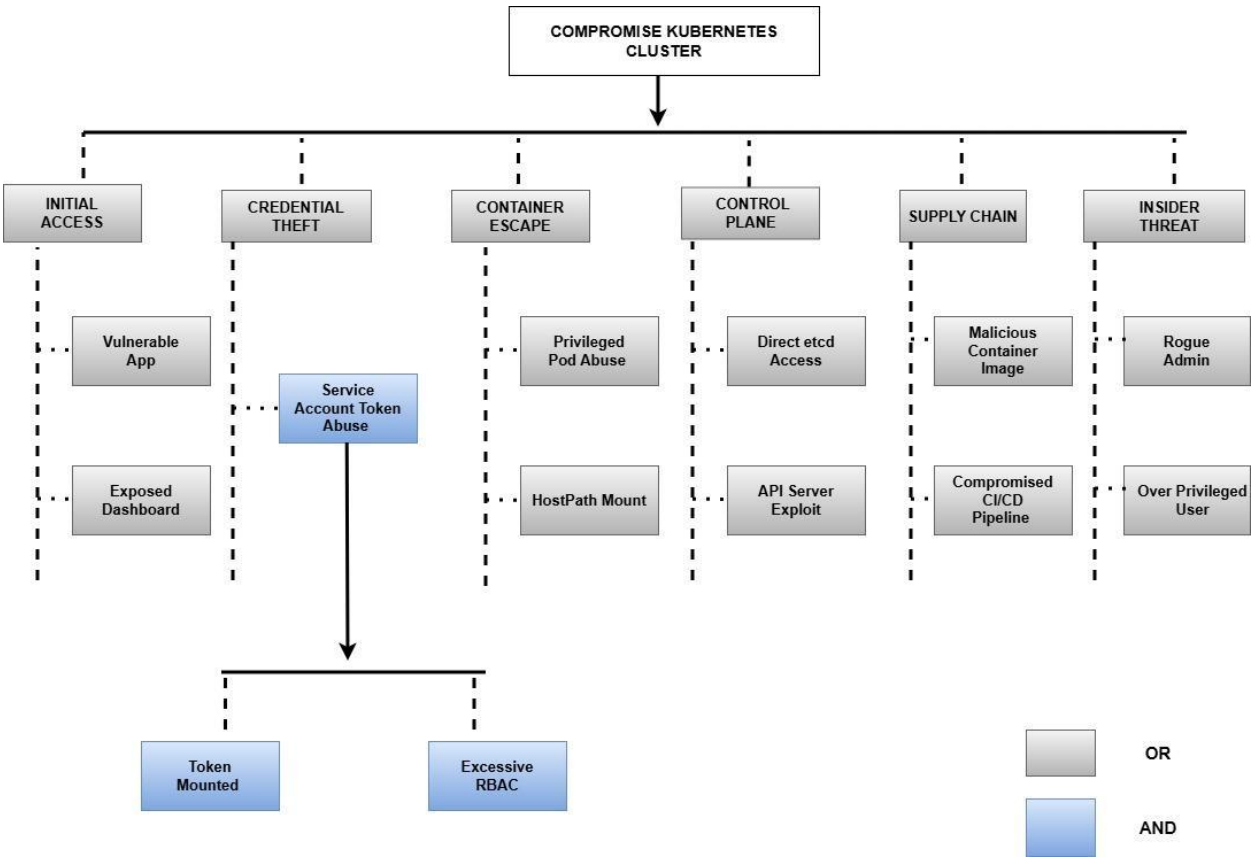

Figure 4. Comprehensive attack tree for Kubernetes cluster compromise with six OR-connected primary branches.

The six branches are:

(1) Initial Access through a vulnerable application or an exposed dashboard, as in the Tesla breach;

(2) Credential Theft requiring both a mounted token AND excessive RBAC permissions connected by AND logic;

(3) Container Escape via privileged pod configurations or hostPath mounts;

(4) Control Plane Attack targeting etcd or the API Server directly;

(5) Supply Chain Attack through malicious container images or a compromised CI/CD pipeline; and

(6) Insider Threat through a rogue administrator or an over-privileged user.

5. Practical Demonstration

This section presents a two-phase demonstration against a deliberately vulnerable Kubernetes cluster (kubernetes-goat). Phase 1 executes the RBAC misconfiguration attack. Phase 2 detects, remediates, and verifies the fix.

5.1 Phase 1: Executing the Attack

The attack begins by creating a pod using the misconfigured superadmin Service Account in the kube-system namespace:

```
kubectl run attacker --image=alpine -n kube-system \
  --overrides='{"spec":{"serviceAccountName":"superadmin"}}' \
  --command -- sleep 3600
```

Figure 5 shows the attack steps: deploying the pod, executing a shell and locating the mounted token, querying the API Server, and decoding the returned secrets from Base64.

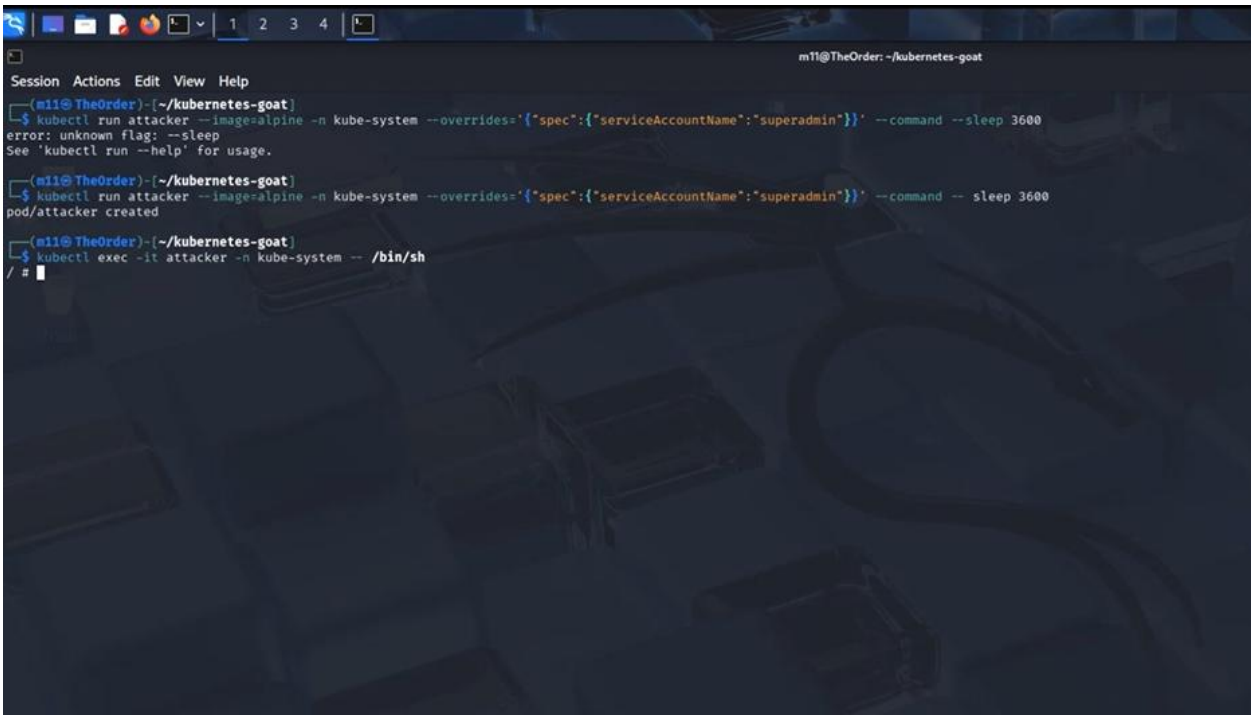

FIgure 5.1: Shell Access

After creating a pod, we, the attacker, then execute an interactive shell inside the container, and from this point forward, we are operating from inside the container with no external tools, only what is available within the container and what Kubernetes automatically provides.

Figure 5.2: Credential Discovery and issuance of authenticated request to the API Server

The first action inside the container is credential discovery. We, as the attacker, navigate the filesystem to locate the automatically mounted Service Account token. With the three files revealed: ca.crt, namespace, and token, we store them in environment variables: APISERVER, TOKEN, and CACERT. You can see that even the API Server address is provided automatically through environment variables. As rhe attacker, we had to discover nothing, every piece of information needed to authenticate was handed to us by default.

Then we issue an authenticated request to the API Server, which returns a complete list of every secret in the cluster, including the bootstrap tokens, authentication credentials, and any application secrets stored in the cluster. We can then decode these secrets from Base64 to obtain usable values, and t this point, we have achieved full cluster compromise, we have read every secret, and because cluster-admin grants all permissions, we could create, modify, or delete any resources. See figure 5.3.

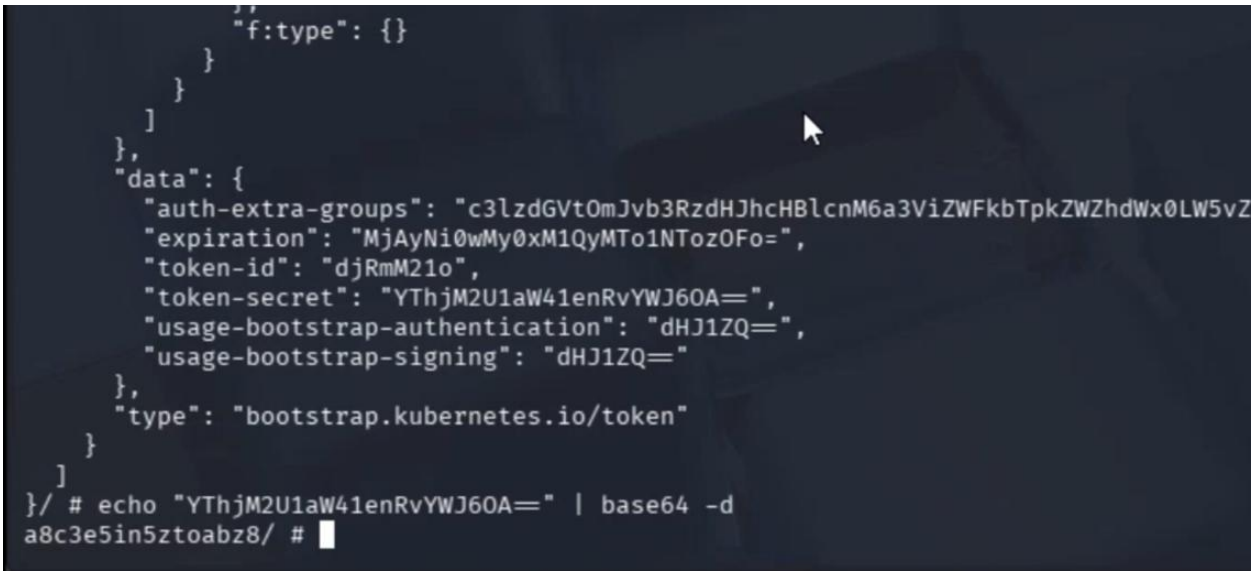

Figure 5.3: Secret decoding and Full Cluster Access.

Every piece of information needed to authenticate against the cluster was provided automatically by default: the Service Account token, the CA certificate, and the API Server address through environment variables. We issued one authenticated request and received the complete list of every cluster secret in plain text. Full cluster compromise was achieved without exploiting any software vulnerability.

5.2 Phase 2: Detection, Remediation, and Verification

The defence proceeds in four steps. kube-bench is deployed to run the CIS Kubernetes Benchmark (Aqua Security, n.d.). The scan returns 57 checks passed, 14 checks failed, and 60 warnings. The dangerous superadmin ClusterRoleBinding is identified, deleted, and replaced with a least-privilege Role granting only pod-read access within the kube-system. Figure 6 shows the defence steps and Figure 7 shows the final result.

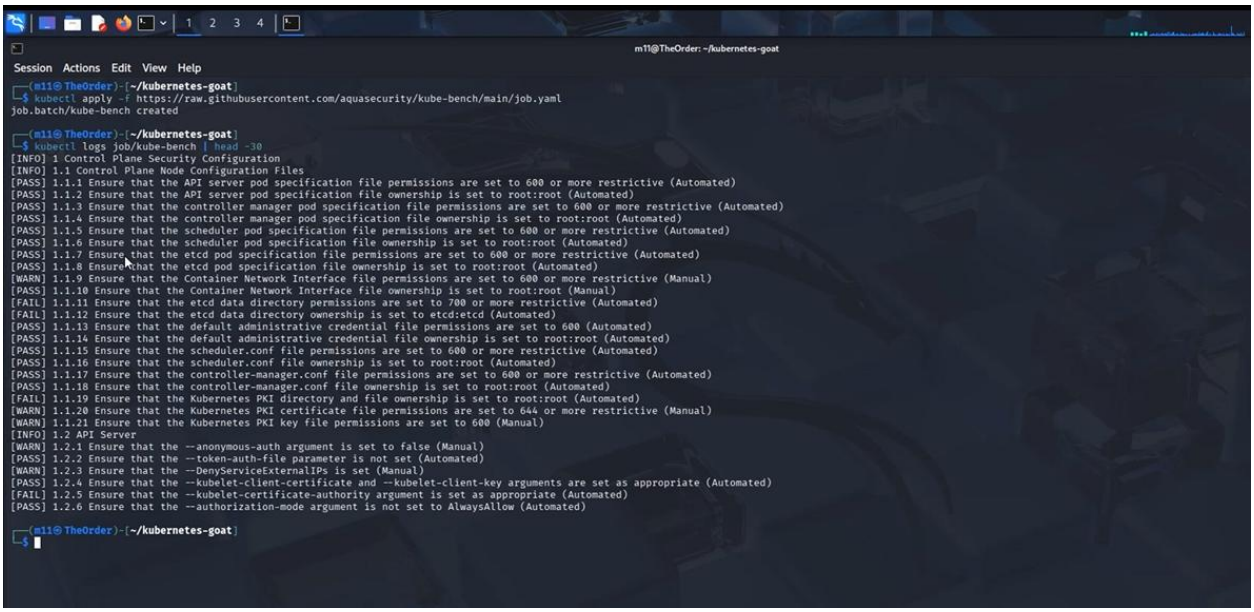

Figure 6.1: kube-bench identifying the misconfiguration

Figure 6.1.1: Scan results

The first step is to identify the misconfiguration. As a defender, we deploy kube-bench, which is an open-source tool that automates evaluation against the CIS Kubernetes Benchmark (a set of industry standard security checks.)

We identify the misconfiguration from the scan result and then confirm that it actually exists. After confirming we delete the overly permissive binding, then create a replacement Role that follows the least privilege principle, which grants only the minimum permissions the Service Account actually needs. In this case, only listing and viewing pods will be permitted. See figure 6.2

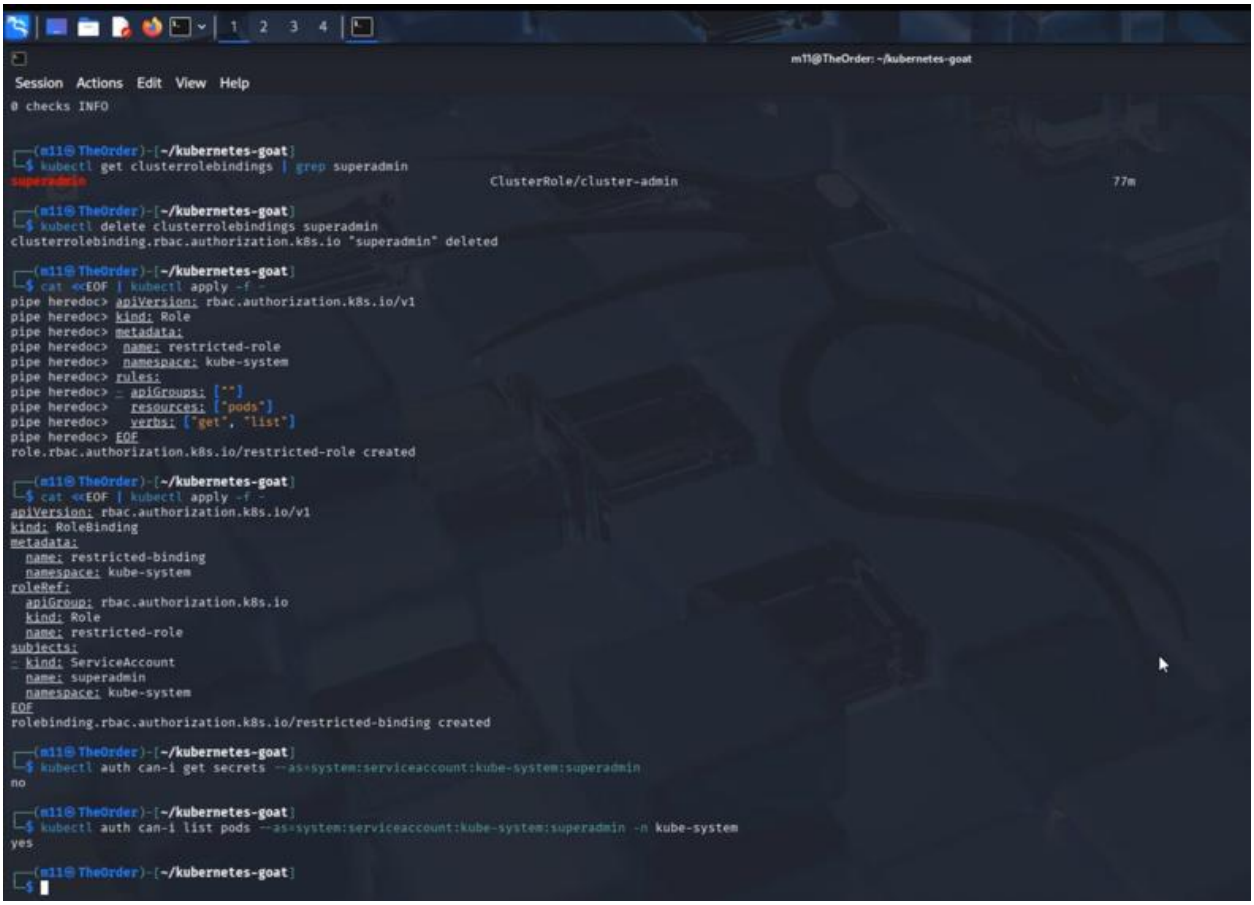

Figure 6.2: Misconfiguration remediation and confirmation

As you can see in the figure above, after creating a role and a corresponding role binding that attaches the restricted role to the Service Account, we verify that it works by using built-in authorization checking.

We can see that this Service Account can no longer access secrets, but it can list and view pods, which are the only permissions it needs.

To fully confirm our fix, as the defender, we repeat the same entire attack sequence. See figures 7 and 7.1 below.

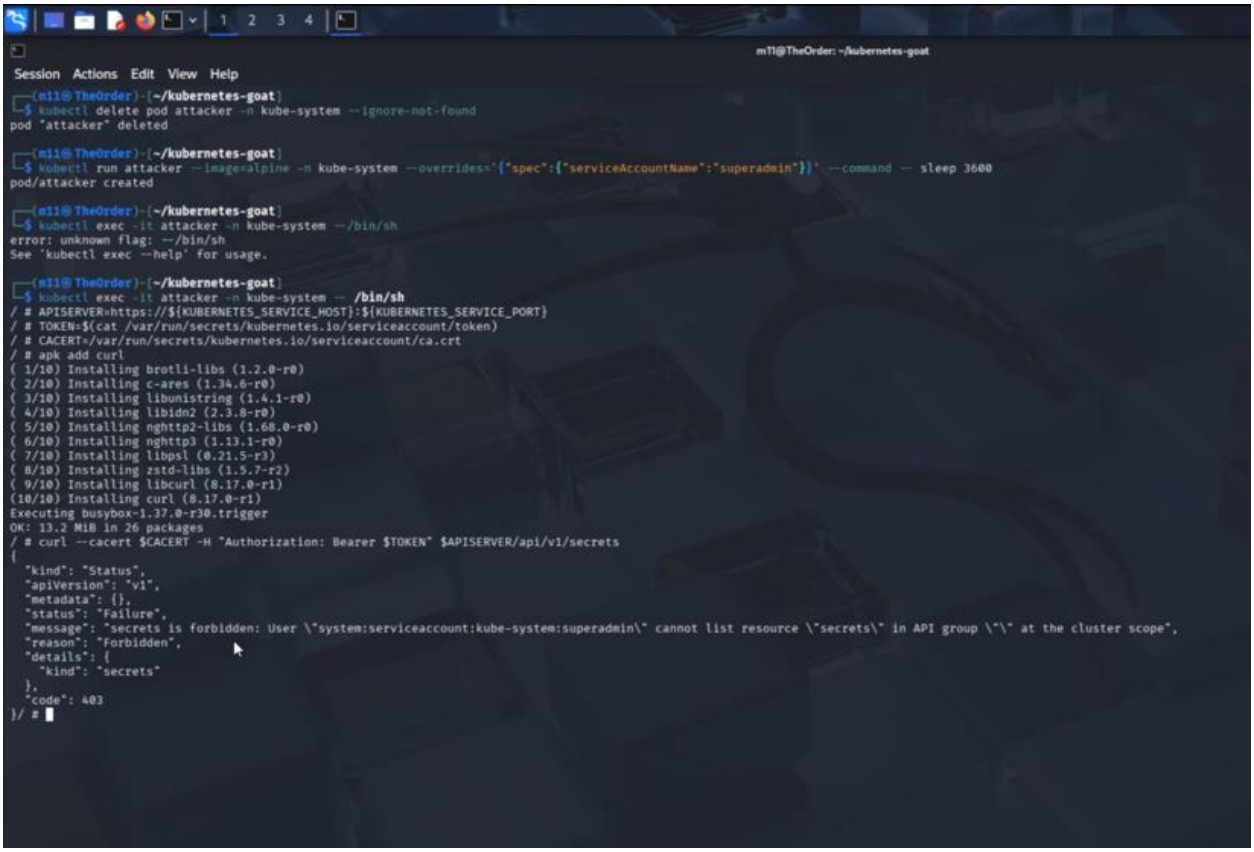

Figure 7: Repeated Attack Sequence

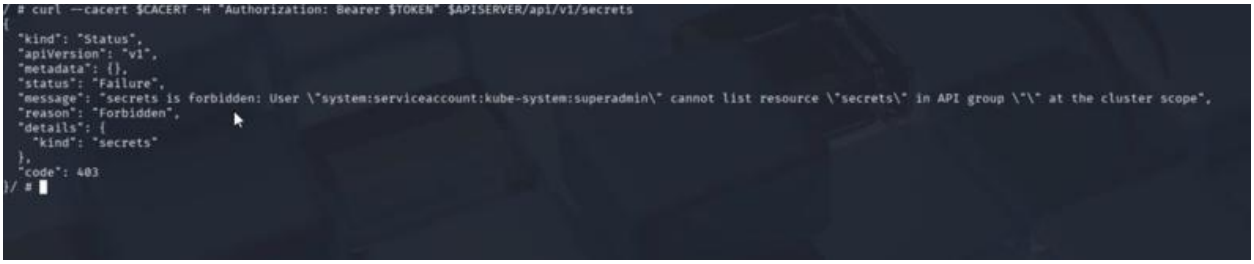

Figure 7.1: Failed Attack

As you can see, this time, the API Server returns a 403 Forbidden response

The token is still mounted. The API Server is still accessible. The curl command is identical. The only change is the RBAC policy, and that single change transforms the outcome from full secret exfiltration to access denied.

5.3 Analysis

The most significant finding is the impact of a single configuration change. The entire attack chain, initial access, credential discovery, API authentication, and secret exfiltration, was broken by modifying one RBAC binding. This validates the analysis from Section 3.5: least privilege, when enforced, blocks not just one step but the entire chain. The attacker still has a shell and a valid token, but the token no longer carries permission to read secrets.

This also illustrates why RBAC misconfiguration falls under the human attack surface rather than the software surface. There is no bug to patch. The software functions correctly. The vulnerability exists purely in a configuration decision made by a human administrator.

6. Security Strategy: Layered Defence Applied to Kubernetes

"Security implementation involves four complementary courses of action: Prevention, Detection, Response, Recovery" (Tavakoli, 2026).

This section applies each layer to Kubernetes and evaluates how they functioned in our demonstration.

6.1 Prevention

Prevention was entirely absent initially. The superadmin Service Account was bound to cluster-admin with no restriction, no Pod Security Standards were enforced, and no network policies restricted the attacker pod. After fixing, a single preventive change, replacing the ClusterRoleBinding with a namespace-scoped least-privilege Role, was sufficient to block the entire attack chain. Had it been configured correctly from the outset, the attack would have been impossible.

6.2 Detection

No detection capability was in place initially. The dangerous ClusterRoleBinding could have existed indefinitely without anyone knowing. Running kube-bench identified 14 failing checks and gave the defender an objective inventory of misconfigurations. In reality, like a production environment, API Server audit logging and Falco runtime monitoring would supplement this, flagging anomalous behaviour like an unexpected shell inside a container or a workload issuing requests to the API Server. If you think about it, detection that occurs before an attack is effectively prevention.

6.3 Response

The response consisted of three actions: identifying the dangerous binding, deleting it to immediately revoke the excessive permissions, and applying a restricted replacement role to restore only the necessary functionality. In a real incident, response would also include isolating affected pods and triggering incident runbooks.

6.4 Recovery

Recovery would require rotating all secrets exposed during the attack: passwords changed, API keys regenerated, and certificates reissued. “Once a secret has been read by an attacker, it must be treated as permanently compromised” regardless of whether the access path has been closed (Shlomi Shaki, 2024).

6.5 Connecting the Four Layers

The demonstration shows a straightforward truth about layered security: the attack succeeded because prevention failed, and continued to succeed because detection was absent. Once CIS scanning introduced detection, the misconfiguration was identified. Once the dangerous binding was deleted and replaced, the attack was neutralised. Recovery would ensure any damage from the initial compromise was contained.

7. Conclusion

This report investigated three questions: how classical security frameworks apply to Kubernetes, what attack paths lead from container to cluster compromise, and how effectively layered security controls can defend against these attacks.

Classical frameworks remain relevant. The CIA triad, threat taxonomy, design principles, and attack tree methodology from academic literature apply directly to modern container orchestration. RBAC misconfiguration enables usurpation, which leads to the disclosure of secrets, precisely as established threat categories predict.

Attack surface analysis reveals systemic risk. The demonstration exploited the human attack surface through misconfiguration, accessed via the software surface through the API, to compromise data assets. The interconnection of attack surfaces amplifies risk.

Design principle violations have direct consequences. The attack succeeded because least privilege was violated. Enforcing this single principle blocked the entire chain.

Defence in depth works. Prevention through RBAC hardening and detection through CIS benchmarking transformed the same attack from full compromise to access denied.

Kubernetes provides robust security mechanisms, but secure defaults are lacking (Reza Ramezanpour, 2025). The gap between secure by design and secure by default remains a challenge. Security is not a product but a process, and the same system, with identical components produced opposite outcomes based solely on configuration.

References

SECTION 1

Docker. (n.d.). What is a container? https://www.docker.com/resources/what-container/

Matt LI. (2025). Top cloud development statistics: Market size and adoption rates. https://www.secondtalent.com/resources/top-cloud-development-statistics-market-size-adoption-rates/

Kubernetes Project. (n.d.). Kubernetes overview. https://kubernetes.io/docs/concepts/overview/

Reza Ramezanpour (2025) Kubernetes is Powerful But Not Secure (at least not by default.) https://www.tigera.io/blog/kubernetes-is-powerful-but-not-secure-at-least-not-by-default/

Sarika Sharma (2025). Cloud misconfigurations causing data breaches. https://fidelissecurity.com/threatgeek/threat-detection-response/cloud-misconfigurations-causing-data-breaches/

ShiftLeft Blog (2025).Inside a Kubernetes Breach: How Threat Actors Exploit Misconfigurations https://medium.com/shiftleft/inside-a-kubernetes-breach-how-threat-actors-exploit-misconfigurations-d03245e84aca

HAProxy Technologies (2025). CVE-2025-59303: secret leak in HAProxy Kubernetes Ingress Controller

https://www.haproxy.com/blog/cve-2025-59303-haproxy-kubernetes-ingress-controller-secret-leak

SECTION 2

Guarino, M. (2026). What is etcd? *Plural Blog*. https://www.plural.sh/blog/what-is-etcd/

Kubernetes Project. (n.d.). Kubernetes cluster architecture. https://kubernetes.io/docs/concepts/architecture/

Kubernetes Project. (n.d.). Kubernetes: Using RBAC Authorization https://kubernetes.io/docs/reference/access-authn-authz/rbac/

Kubernetes Project. (n.d.). Kubernetes: Ports and Protocols https://kubernetes.io/docs/reference/networking/ports-and-protocols/

Kubernetes Project. (n.d.). Admission Controller in Kubernetes https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

SECTION 3

Kubernetes Project. (n.d.). Kubernetes: kube-apiserver https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/

Kubernetes Project. (n.d.). Kubernetes: Secrets https://kubernetes.io/docs/concepts/configuration/secret/

Kubernetes Project. (n.d.). Kubernetes: Network Policies

https://kubernetes.io/docs/concepts/services-networking/network-policies/

Kubernetes Project. (n.d.). Kubernetes: Container Runtimes

https://kubernetes.io/docs/setup/production-environment/container-runtimes/

Kubernetes Project. (n.d.). Kubernetes: Official CVE Feed

https://kubernetes.io/docs/reference/issues-security/official-cve-feed/

SECTION 4

Microsoft. (n.d.). *Threat Matrix for Kubernetes*. https://microsoft.github.io/Threat-Matrix-for-Kubernetes/

MITRE Corporation. (n.d.). ATT&CK Matrix: Enterprise, Containers. https://attack.mitre.org/matrices/enterprise/containers/

SECTION 5

Kubernetes Project. (n.d.). Kubernetes: Security. https://kubernetes.io/docs/concepts/security/

Kubernetes Project. (n.d.). Kubernetes:Configure Service Account for Pods https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/

Kubernetes Project. (n.d.). Kubernetes: Securing a Cluster https://kubernetes.io/docs/tasks/administer-cluster/securing-a-cluster/

Akula, M. (n.d.). *Kubernetes Goat*. GitHub. https://github.com/madhuakula/kubernetes-goat

Aqua Security. (n.d.). *kube-bench*. GitHub. https://github.com/aquasecurity/kube-bench

Center for Internet Security. (2025). *CIS Kubernetes Benchmark v1.12.0*. CIS.

SECTION 6

Kubernetes Project. (n.d.) Kubernetes: Pod Security Admission https://kubernetes.io/docs/concepts/security/pod-security-admission/

Kubernetes Project. (n.d.). Kubernetes: Auditing https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/

Shlomi Shaki (2025) Secret-Remediation: The true cost of remediating leaked secrets in code https://sentry01.github.io/Secret-Remediation/

DEMO VIDEO

https://drive.google.com/drive/folders/1ElWSIRTQGNx_8IGLG3h_uGNW_lYfpf1t

Tavakoli, Y. (2026). *COMP 3019: Security and privacy in computer systems* [Course lecture notes]. Memorial University of Newfoundland.